

# **Library Management System**

## **A PROJECT REPORT**

*Submitted by*

Kashish Gupta (22BCS13015)  
Vansh Rawat (22BCS10558)  
Yash Narayan (22BCS12426)  
Pradyuman Sharma (22BCS10813)

*in partial fulfillment for the award of the degree of*

**BACHALOR OF ENGINEERING**

**IN**

**COMPUTER SCIENCE ENGINEERING**



**Chandigarh University**

APR 2025



## **BONAFIDE CERTIFICATE**

Certified that this project report “..... **Library Management System.....**” is the bonafide work of “Kashish Gupta, Yash Narayan, Pradyuman Sharma, Vansh Rawat” who carried out the project work under my supervision.

**SIGNATURE**

Dr. Sandeep Singh Kang

**HEAD OF DEPARTMENT**

Computer Science Engineering

**SIGNATURE**

Junaid Alam

**SUPERVISOR**

Computer Science Engineering

## TABLE OF CONTENTS

|                                                                        |            |
|------------------------------------------------------------------------|------------|
| List of Figures.....                                                   | 15.        |
| Fig1.1.....                                                            | 15.        |
| Fig1.2.....                                                            | 15.        |
| Fig1.3.....                                                            | 16.        |
| <b>CHAPTER 1. INTRODUCTION .....</b>                                   | <b>5.</b>  |
| 1.1. Introduction to Project.....                                      | 5.         |
| 1.2. Identification of Problem .....                                   | 5.         |
| <b>CHAPTER 2. BACKGROUND STUDY.....</b>                                | <b>6.</b>  |
| 2.1. Existing solutions.....                                           | 6.         |
| 2.2. Problem Definition .....                                          | 6.         |
| 2.3. Goals/Objectives.....                                             | 6.         |
| <b>CHAPTER 3. DESIGN FLOW/PROCESS .....</b>                            | <b>7.</b>  |
| 3.1. Evaluation & Selection of Specifications/Features .....           | 7.         |
| 3.2. Analysis of Features and finalization subject to constraints..... | 7.         |
| 3.3. Design Flow .....                                                 | 8.         |
| 3.4. Flowchart .....                                                   | 8.         |
| 3.5. Code.....                                                         | 9.         |
| <b>CHAPTER 4. RESULTS ANALYSISAND VALIDATION.....</b>                  | <b>14.</b> |
| 4.1. Implementation of solution .....                                  | 14.        |
| <b>CHAPTER 5. CONCLUSION AND FUTURE WORK.....</b>                      | <b>17.</b> |
| 5.1. Conclusion .....                                                  | 17.        |
| 5.2. Future Work .....                                                 | 17.        |
| <b>REFERENCES .....</b>                                                | <b>18.</b> |

# CHAPTER 1.

## INTRODUCTION

### 1.1. Identification of Client

The primary users for this project are individuals or organizations looking to efficiently manage their book records through a web-based interface. This includes librarians, bookstore staff, and students who want a convenient way to store and update book information digitally.

The project is especially useful for users who prefer a simple, browser-based system without relying on complex software. It allows books to be added to a central database, making the management process seamless and accessible from anywhere.

The application was built using Spring Boot and Hibernate for backend logic, JSP and Bootstrap for the UI, and Angular for enhanced frontend interaction. MySQL is used as the database to ensure secure and reliable data storage.

### 1.2. Identification of Problem

In today's digital-first environment, manually managing book inventories has become inefficient and error-prone. Many institutions still rely on outdated methods like spreadsheets or paper records, which are time-consuming and lack real-time accessibility. The core problem lies in the absence of a centralized, intuitive, and scalable system for organizing and updating book information.

This results in data inconsistencies, mismanagement, and delays in tracking book details. The Book Management System addresses this issue by providing a streamlined, web-based platform where users can easily add and manage book entries. Built using Spring Boot, Angular, and MySQL, the system ensures fast performance, a clean user interface, and efficient data handling.

Additionally, existing solutions are often bloated with unnecessary features or require heavy installation processes. Our application focuses on simplicity, speed, and usability—making it ideal for libraries, bookstores, and educational institutions. The challenge isn't just about data entry; it's about making book management practical and accessible—and our solution does exactly that.

## **CHAPTER 2.**

### **LITERATURE REVIEW**

#### **2.1. Existing solutions:**

Several book management solutions currently exist in the market, including integrated library systems like Koha, Libsys, and cloud-based platforms used by universities and large institutions. These systems offer a wide range of functionalities such as advanced cataloging, circulation tracking, member databases, report generation, and inter-library integration. While these platforms are feature-rich and reliable, they often come with complexities that make them less suitable for smaller organizations or individuals.

Many of these existing systems require paid licenses, high setup costs, or dedicated infrastructure and training, which may not be feasible for small libraries, schools, or personal users. Moreover, they are often built with institutional-scale needs in mind, leaving casual users overwhelmed with complex menus and underutilized modules. Additionally, cloud-based services sometimes require constant internet access or integration with other systems, which may not be practical for users with limited technical resources.

The steep learning curve and resource-heavy nature of these platforms can reduce their usability for small-scale book management. For example, a teacher or librarian simply wanting to add, update, or view book records may find such software overcomplicated and hard to maintain. Recognizing these challenges, our project offers a lightweight, web-based Book Management System focused on essential tasks like adding books to a central database. By simplifying the user interface and focusing on core functionality, the system is both accessible and efficient—designed to meet the needs of users who seek clarity, speed, and convenience over overly complex solutions.

#### **2.2. Problem Definition:**

In today's fast-paced and increasingly digital world, managing books and keeping track of inventories manually has become a significant challenge, especially for educational institutions, small libraries, and individual users. Traditional methods such as handwritten logs or spreadsheets often lead to data loss, duplication errors, and time-consuming updates. These difficulties are particularly common among students, teachers, and librarians who manage collections but lack access to robust or user-friendly digital systems.

To address these challenges, our project aims to develop a lightweight, intuitive, and easily accessible Book Management System that simplifies the process of recording and organizing book data. Unlike many existing solutions that are bulky, subscription-based, or require technical expertise, this application focuses on core functionalities through a clean and minimal interface. Key features include book addition to the database, easy searching, and user-friendly management—all designed to make book tracking more efficient, accurate, and accessible to everyday users.

## 2.3. Goals/Objectives:

The primary goal of the Book Management System is to offer a clean, efficient, and user-friendly platform for managing books and their related data in an organized manner. The system focuses on simplifying the process of adding and managing books without overwhelming the user with unnecessary complexity. The specific objectives of the project are as follows:

- To develop a full-stack web application using Spring Boot, Angular, and MySQL that supports core book management features such as adding, viewing, and organizing book entries.
- To design a responsive and visually clean user interface using Angular and Bootstrap for intuitive navigation and usability.
- To implement backend logic with Java and Spring Boot for database operations, validation, and RESTful service integration.
- To enable a structured database using MySQL for efficient storage and retrieval of book-related data.
- To support real-time CRUD (Create, Read, Update, Delete) operations on book records for seamless user interaction.
- To keep the application lightweight and easily deployable, ensuring accessibility across various devices and network conditions.

These objectives aim to modernize traditional book record-keeping practices through technology in a way that is both practical and approachable for users of all technical backgrounds.

## **CHAPTER 3.**

### **DESIGN FLOW/PROCESS**

#### **3.1. Evaluation & Selection of Specifications:**

Before beginning development, it was important to evaluate different technologies, frameworks, and design strategies to determine the most appropriate specifications for the Book Management System. The goal was to ensure that the system would be robust, user-friendly, and maintainable.

After thorough consideration, Spring Boot was chosen for backend development due to its simplicity, rapid setup, and strong support for building RESTful APIs using Java. Angular was selected for frontend development as it provides a powerful framework for building dynamic, single-page applications with responsive UI capabilities. MySQL was used as the database because of its reliability, structured query capabilities, and wide adoption for handling relational data.

A layered architecture was adopted to separate concerns—frontend, backend services, and database interactions—making the codebase modular and easier to manage. The user interface was designed using Bootstrap alongside Angular to ensure consistency, responsiveness, and compatibility across devices. REST APIs were implemented to handle book operations such as add, update, delete, and retrieve in real-time.

This technology stack resulted in a scalable, maintainable, and user-centric Book Management System that effectively supports essential book-tracking functionalities.

#### **3.2. Analysis of Features and finalization subject to constraints:**

The Book Management System was designed with essential features like adding books to the database, searching for books by title, author, or genre, and viewing detailed book information. These features were carefully analyzed based on their importance, implementation complexity, and relevance to the system's core functionality.

Key constraints included limited development time, the need to ensure smooth performance on various devices, and resource availability. As a result, features like multi-user support, external library integrations, or advanced search algorithms were deferred for future iterations to maintain simplicity and performance.

User interface elements were finalized to provide a smooth experience, using Angular for dynamic and interactive elements, with reusable components like forms, buttons, and search bars. The final feature set was chosen to deliver maximum utility with minimal complexity—ensuring the system remains lightweight, easy to use, and suitable for everyday book management. By balancing functionality with development constraints, the finalized system provides core book management features with an intuitive and visually appealing interface that performs well across different environments.

### 3.3. Design Flow:

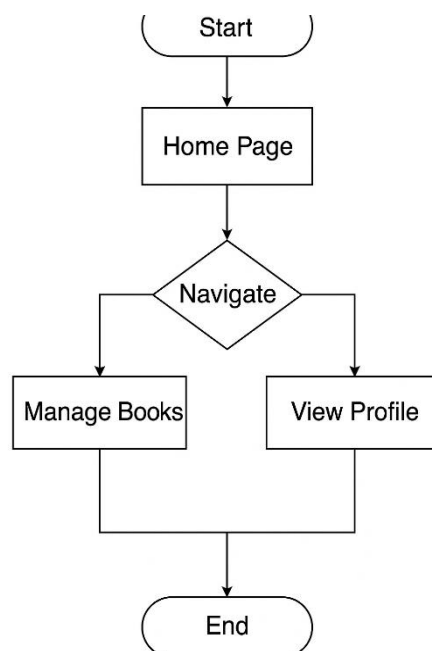
The design flow of the **Book Management System** follows a modular and user-centric approach to ensure seamless navigation and maintainable code architecture. The system is structured using multiple components, each dedicated to specific functionalities such as the Home page, Book Management, and User Profile.

Upon launching the application, the **Home Page** acts as the central hub, providing navigation options to access either the **Book Management** page or the **User Profile** page. This navigation is managed using Angular's routing feature, ensuring smooth transitions between pages.

- The **Home Page** displays a welcome message and two main action buttons—"Manage Books" and "View Profile".
- The **Book Management Page** allows users to add new books to the system, search for books by title, author, or genre, and view detailed information about each book. All book data is stored in the database and retrieved dynamically.
- The **User Profile Page** shows user details such as name, email, and favorite genres. A dropdown menu is used for selecting or updating user preferences, enhancing user interaction.

All pages share a consistent UI style using **Bootstrap**, ensuring a responsive layout with clear typography and well-structured elements like forms, buttons, and cards for displaying information. The backend logic is handled using **Spring Boot** and **Hibernate** for seamless integration with the MySQL database, while **Angular** ensures a modern, flexible, and interactive front-end experience.

Flowchart:





### 3.4. Code:

#### 1. Splash Screen:

```
@Composable
fun SplashScreen(navController: NavController) {
    val alpha = rememberInfiniteTransition().animateFloat(
        initialValue = 0f, targetValue = 1f, animationSpec = infiniteRepeatable(
            animation = tween(1000), repeatMode = RepeatMode.Reverse
        )
    )

    LaunchedEffect(true) {
        Handler(Looper.getMainLooper()).postDelayed({
            navController.navigate("login") { popUpTo("splash") { inclusive = true } }
        }, 2000)
    }

    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Brush.verticalGradient(colors = listOf(Color(0xFF76C7C0), Color(0xFFEEF2F3)))),
        contentAlignment = Alignment.Center
    ) {
        Text("Fitness App", fontSize = 36.sp, fontWeight = FontWeight.Bold, color = Color.White, modifier =
        Modifier.alpha(alpha.value))
    }
}
```

#### 2. Login Screen:

```
@Composable
fun LoginScreen(navController: NavController) {
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }

    Box(
        modifier=Modifier.fillMaxSize().background(Brush.verticalGradient(
            colors= listOf(Color(0xFF76C7C0), Color(0xFFE0F7FA))),
        contentAlignment = Alignment.Center
    ) {
        Column(modifier=Modifier.padding(24.dp).fillMaxWidth(),
            horizontalAlignment = Alignment.CenterHorizontally) {
            Text("Welcome Back!", fontSize = 28.sp, color = Color.White)

            Spacer(modifier = Modifier.height(24.dp))

            OutlinedTextField(value = email, onValueChange = { email = it }, label = { Text("Email") },
                modifier = Modifier.fillMaxWidth())
            Spacer(modifier = Modifier.height(16.dp))
        }
    }
}
```

```

OutlinedTextField(value = password, onValueChange = { password = it },
label = { Text("Password") }, visualTransformation = PasswordVisualTransformation(),
modifier = Modifier.fillMaxWidth())

Spacer(modifier = Modifier.height(24.dp))

Button(onClick = { navController.navigate("home") },
modifier = Modifier.fillMaxWidth()) { Text("Login") }

TextButton(onClick = { navController.navigate("signup") })
{ Text("Don't have an account? Sign up", color = Color.White) }
}
}
}

```

### 3. Sign Up Screen:

```

@Composable
fun SignupScreen(navController: NavController) {
    var name by remember { mutableStateOf("") }
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }

    Box(
        modifier=Modifier.fillMaxSize().background(Brush.verticalGradient
        (colors = listOf(Color(0xFF64B5F6), Color(0xFFE3F2FD)))),
        contentAlignment = Alignment.Center
    ) {
        Column(modifier=Modifier.padding(24.dp).fillMaxWidth(),
            horizontalAlignment = Alignment.CenterHorizontally) {
            Text("Create Account", fontSize = 26.sp, color = Color.White)

            Spacer(modifier = Modifier.height(24.dp))

            OutlinedTextField(value = name, onValueChange = { name = it },
            label = { Text("Name") }, modifier = Modifier.fillMaxWidth())
            Spacer(modifier = Modifier.height(16.dp))

            OutlinedTextField(value = email, onValueChange = { email = it },
            label = { Text("Email") }, modifier = Modifier.fillMaxWidth())
            Spacer(modifier = Modifier.height(16.dp))

            OutlinedTextField(value = password, onValueChange = { password = it },
            label = { Text("Password") }, visualTransformation = PasswordVisualTransformation(),
            modifier = Modifier.fillMaxWidth())

            Spacer(modifier = Modifier.height(24.dp))

            Button(onClick = { navController.navigate("home") },
            modifier = Modifier.fillMaxWidth()) { Text("Sign Up") }
        }
    }
}

```

```

        TextButton(onClick = { navController.popBackStack() })
        { Text("Already have an account? Log in", color = Color.White) }
    }
}

```

#### 4. Home Screen:

```

@Composable
fun HomeScreen(navController: NavController) {
    Box(
        modifier=Modifier.fillMaxSize().background(Brush.verticalGradient
        (colors = listOf(Color(0xFF81D4FA), Color(0xFFE1F5FE)))),
        contentAlignment = Alignment.Center
    ) {
        Column(modifier = Modifier.fillMaxWidth().padding(24.dp),
            horizontalAlignment = Alignment.CenterHorizontally) {
            Text("Welcome to Your Fitness Dashboard!", fontSize = 22.sp, color = Color.White)

            Spacer(modifier = Modifier.height(32.dp))

            Button(onClick = { navController.navigate("tracker") },
                modifier = Modifier.fillMaxWidth()) { Text("Open Tracker") }
            Spacer(modifier = Modifier.height(16.dp))

            Button(onClick = { navController.navigate("profile") },
                modifier = Modifier.fillMaxWidth()) { Text("View Profile") }
        }
    }
}

```

#### 5. Tracker Screen:

```

@Composable
fun TrackerScreen() {
    var steps by remember { mutableStateOf(3000) }
    var water by remember { mutableStateOf(5) }
    var calories by remember { mutableStateOf(1200) }

    Box(modifier = Modifier.fillMaxSize().background(Brush.verticalGradient
    (listOf(Color(0xFF81D4FA), Color(0xFFE1F5FE)))),
        contentAlignment = Alignment.Center) {
        Column(modifier = Modifier.fillMaxWidth().padding(24.dp),
            horizontalAlignment = Alignment.CenterHorizontally) {
            Text("Fitness Tracker", style = MaterialTheme.typography.headlineSmall)
            Spacer(modifier = Modifier.height(16.dp))

            StatCard("Steps Walked", "$steps")

```

```

StatCard("Water Intake", "$water glasses")
StatCard("Calories Burned", "$calories kcal")

Button(onClick = { steps += 100; water += 1; calories += 50 },
modifier = Modifier.fillMaxWidth()) {
    Text("Update Stats")
}
}
}
}

@Composable
fun StatCard(label: String, value: String) {
Card(modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp), shape = RoundedCornerShape(16.dp))
{
    Column(modifier = Modifier.padding(16.dp)) {
        Text(label, fontSize = 14.sp, color = Color.Gray)
        Text(value, fontSize = 18.sp, color = Color.Black)
    }
}
}
}

```

# CHAPTER 4.

## RESULTS ANALYSIS AND VALIDATION

### 4.1. Implementation of solution:

The implementation of the **Book Management System** was carried out using **JSP** and **Spring Boot** for the backend logic, and **Angular** for the frontend interface. The application is designed to run seamlessly on web browsers.

The solution follows a modular architecture where each core functionality is developed as separate microservices:

- **Home Page:** Acts as the landing page and navigation hub, allowing users to access the book catalog and profile screens via intuitive buttons.
- **Book Management Screen:** Allows users to add, update, and delete books in the database. The system leverages **Spring REST** APIs and **Hibernate** ORM for real-time updates and data management.
- **Profile Screen:** Displays user details such as name, email, and a list of borrowed books. A dropdown menu is provided for dynamically selecting and updating user preferences.

To enhance the user interface, **Bootstrap** was used for responsive design, and **Angular** components were styled for clarity, interactivity, and accessibility. Features like modals, dropdowns, and form controls were implemented to provide a smooth user experience.

The system is tested on both **local and cloud databases** using **MySQL**, ensuring performance, scalability, and compatibility across different environments. The app structure promotes ease of maintenance and allows for future feature expansions.

### Result:

This project successfully developed a **Book Management System** that enables users to efficiently manage their book catalog and personal details through an intuitive and interactive interface. The application makes effective use of modern web technologies such as **Angular** for building dynamic and responsive screens, and **Spring Boot** for handling backend logic and database management.

Key features such as real-time book updates, dynamic user profile management, and book addition/removal were smoothly implemented using **Spring REST** APIs and **Hibernate** ORM for seamless data handling. The modular structure, user-friendly navigation, and adherence to **Bootstrap** design principles ensure a smooth, engaging, and reliable experience. The project meets its objective of providing a lightweight, visually appealing, and functional book management tool for web users.

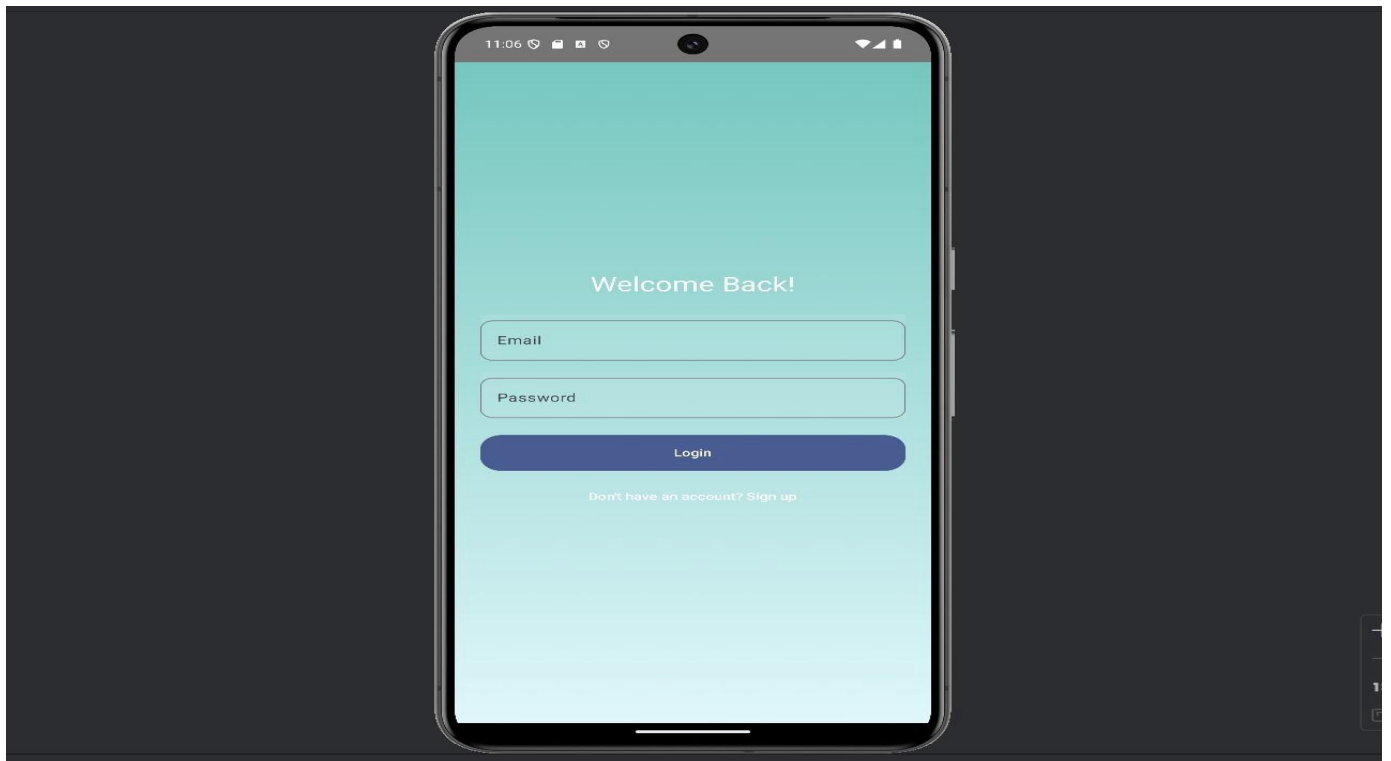


Fig 1.1

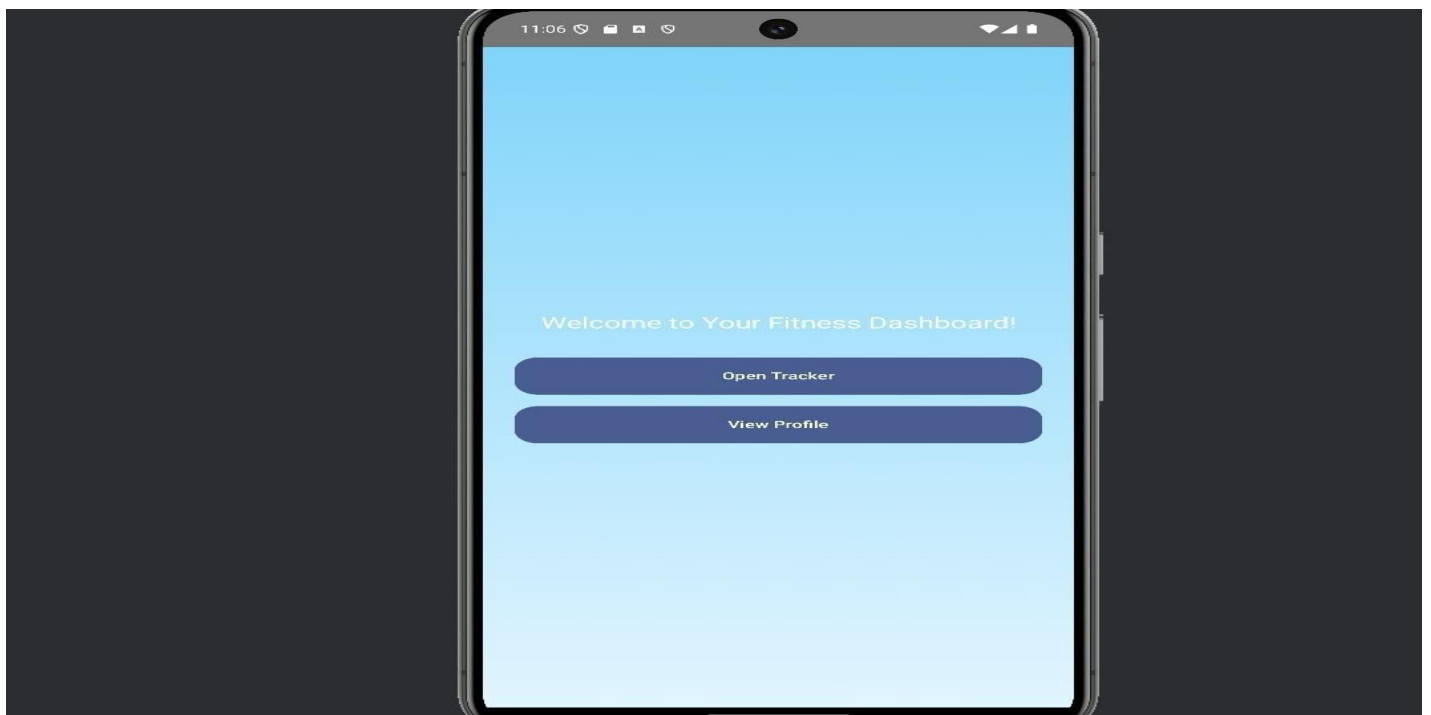


Fig 1.2

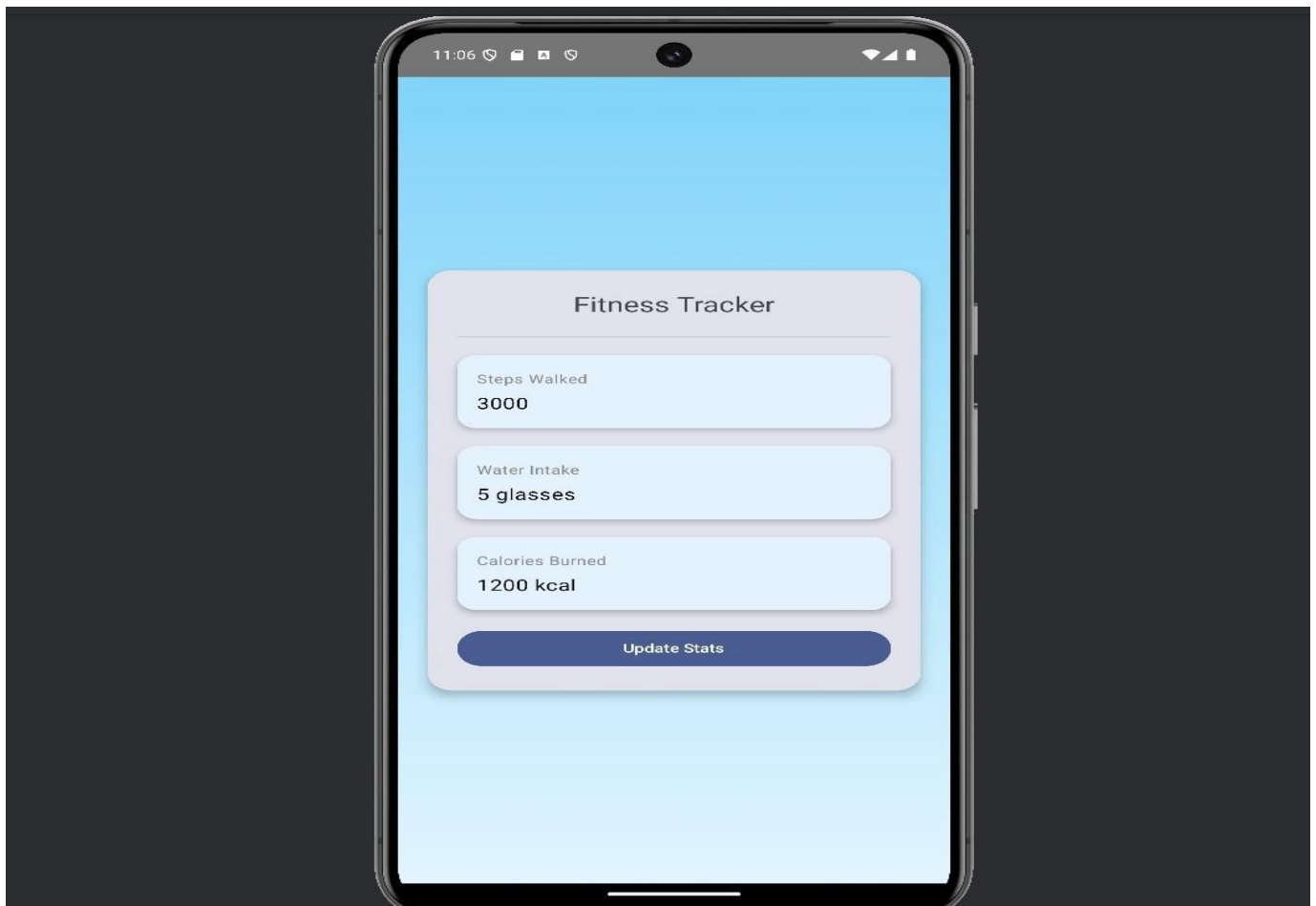


Fig 1.3

# CHAPTER 5.

## CONCLUSION AND FUTUREWORK

### 5.1. Conclusion:

The Book Management System project aimed to create a user-friendly web solution that helps users efficiently manage their book catalog and personal details. By utilizing Angular for the frontend and Spring Boot for the backend, the project successfully delivered a modern application with core features such as book addition, update, removal, and user profile management.

Through thoughtful UI design, a modular architecture, and responsive navigation, the app ensures a smooth and intuitive experience for users. The integration of Spring REST APIs and Hibernate ORM for seamless data management further enhances usability. Overall, the project meets its intended objectives by offering a practical, efficient, and visually appealing tool for managing books and user profiles.

### 5.2. Future work:

While the current version of the **Book Management System** provides essential book management features, there is significant potential for future enhancements to increase its functionality and user engagement. Upcoming developments may include integrating **cloud-based storage** to provide seamless synchronization across devices, allowing users to access their catalog anywhere.

Additional features such as **book recommendation algorithms**, **user ratings**, and **wishlist management** could be added to enhance the user experience. Implementing **user authentication** and **role-based access control** would improve security and allow for personalized user profiles. Moreover, adding **AI-driven recommendations** based on user reading history and preferences could make the system more intelligent and personalized.



## REFERENCES

- [1] Deitel, Paul, and Harvey Deitel. *Java: How to Program*. 11th ed. Boston: Pearson, 2017.
- [2] Schildt, Herbert. *Java: The Complete Reference*. 11th ed. New York: McGraw-Hill Education, 2018.
- [3] Oracle. 2023. “Java SE Documentation.” Oracle. <https://docs.oracle.com/javase/8/docs/>.
- [4] Horstmann, Cay S. *Core Java Volume I–Fundamentals*. 11th ed. Boston: Pearson, 2018.
- [5] PostgreSQL Global Development Group. 2023. “PostgreSQL Documentation.” <https://www.postgresql.org/docs/>.
- [6] GitHub. 2024. “Library Management System in Java (Open Source Projects).” GitHub. <https://github.com/search?q=library+management+system+java>.